

Interdisciplinary Senior Capstone Project

RoboNeuro Mapping

Team 47

Qindong Gan, Wentao Zhang , Qiyanhui Lu

Fall 2024

2024-2025 Academic Year

EECE 5032 EE SENIOR DES II (001)

Faculty Advisor: Dr. Ali Minai (EECS)

Course Instructor: Dr. Wei Wei (EECS)

April 21th, 2025

A Design Project Final Report submitted to the
Department of Electrical Engineering and Computer Science (EECS)
College of Engineering and Applied Science
University of Cincinnati
In partial fulfillment of the requirements for the degree of
Bachelor of Science

Team member's Signature

Technical Advisors Signature

Acknowledgements

We would like to sincerely thank Dr. Ali Minai for his valuable insights and continued guidance throughout the development of this project. His support and mentorship played a crucial role in shaping our research direction and ensuring steady progress.

We are also grateful to Andrew Gerstenslager and Bekarys Dukenbaev from Dr. Minai's lab, whose previous work laid the foundation for this project. Their improvements to the earlier model and the assistance they offered during our simulation process were instrumental to our success.

Lastly, we deeply appreciate the resources and support provided by the Department of Electrical Engineering and Computer Science at the University of Cincinnati, without which this project would not have been possible.

Table of Contents

1	Abstract.....	1
2	Introduction.....	1
2.1	Background.....	1
2.2	Problem.....	2
2.3	Solution.....	2
2.4	Project Goals.....	2
2.5	Credibility.....	3
3	Discussion.....	4
3.1	Project Concept.....	4
3.2	Design Objectives.....	4
3.2.1	Hardware:.....	4
3.2.2	Software:.....	5
3.3	Technical Approach.....	5
3.3.1	Basic Theory.....	5
3.3.2	Proposed Solution.....	9
3.3.3	Design Alternative.....	14
3.3.4	Design Considerations.....	16
3.3.5	Design Constraints Identification.....	17
3.3.6	Engineering standards and Codes Applied to the Project.....	17
3.3.7	Process.....	18
3.3.8	Performance.....	19
3.4	Timeline.....	27
3.5	Problems Encountered / Analysis of Problems Solved.....	29
3.6	Future Recommendations.....	31
4	Conclusion.....	32
	References.....	33

1 Abstract

This project enhances a biologically inspired navigation framework by integrating visual perception into a hippocampal-based spatial learning model. Building upon a prior system that relied solely on LiDAR-derived boundary information, we incorporated high-dimensional image embeddings extracted using EfficientNetV2B0 and fused them with LiDAR input via an autoencoder-based fusion layer. The combined features are processed through a place cell network to generate a cognitive map for autonomous navigation. All experiments were conducted in Webots R2025 across both simple and visually aliased environments. Results show that while visual inputs offer marginal improvements in spatial representation smoothness and place field activation, challenges such as perceptual aliasing persist in symmetric layouts. Our work demonstrates the potential and limitations of vision-LiDAR fusion and proposes future directions involving spatially aware vision models and contrastive fusion strategies.

2 Introduction

2.1 Background

Our project builds upon a biologically inspired navigation model, which simulated spatial cognition mechanisms in mammals using place cells, boundary vector cells (BVCs), and head direction cells (HDs) [1][2].

In the original model, BVCs used 360-degree LiDAR input to detect distances to surrounding boundaries, producing a 720-dimensional input vector. HD cells, represented by eight neurons, encoded the robot's orientation based on its heading direction. These components together provided a spatial frame of reference. The place cell network (PCN) consisted of neurons that

responded to specific locations, organizing themselves through a winner-take-all mechanism and synaptic plasticity. This allowed the robot to autonomously form place fields that covered the environment uniformly and adaptively [3][4][5].

2.2 Problem

Traditional SLAM systems, though powerful, face critical limitations including high computational overhead, difficulty handling environmental symmetry, and poor adaptability to dynamic scenes. Bio-inspired models offer a compelling alternative by emulating mammalian neural structures such as place cells, head direction cells, and boundary vector cells [5]. However, prior implementations relied solely on LiDAR input, resulting in limited perception diversity and an inability to resolve perceptually similar but spatially distinct environments [3].

2.3 Solution

Our project addresses these limitations by augmenting an existing biologically inspired place-cell model with a visual sensory pathway. Specifically, we integrate CNN-derived image embeddings into the navigation model alongside LiDAR-based spatial representations. A dedicated fusion layer compresses these multi-modal features into a unified state vector that serves as input to the place cell network. The enhanced model is tested in the simulated environment which is Webots R2025 with environments designed to evaluate performance under both simple and ambiguous spatial conditions [6][7].

2.4 Project Goals

- Develop a robust, biologically inspired navigation framework capable of integrating LiDAR and vision inputs.
- Improve place cell activation patterns and reward propagation in ambiguous environments.

- Evaluate how visual context affects spatial representations, especially in symmetric or aliased scenes [5][6].

2.5 Credibility

Qindong Gan played a central role in group coordination and workflow management, ensuring that tasks were clearly assigned, and deadlines were met. He was responsible for organizing weekly meetings, preparing project update reports, and leading literature reviews in the early phase. In addition, Qindong contributed significantly to the vision embedding module, including model training, feature extraction, and performance evaluation.

Wentao Zhang focused on the design and implementation of the overall navigation model. He was primarily responsible for integrating the biologically inspired components—place cells, boundary vector cells, and head direction cells—into a unified architecture. Wentao also led the development of the fusion layer and contributed to comparative testing between vision-enabled and LiDAR-only configurations.

Qiyanhui Lu contributed across multiple technical components, particularly in optimizing the fusion mechanism and analyzing model performance under different environmental settings. He was also involved in debugging simulation pipelines, preparing visualizations, and managing version control for collaborative code development. In team discussions, Qiyanhui frequently summarized results and guided decisions regarding model architecture refinements.

3 Discussion

3.1 Project Concept

The project aims to extend a hippocampal-inspired navigation model by embedding visual perception into its spatial learning process. The original model relied solely on LiDAR to simulate boundary vector cells and used a reinforcement learning mechanism to map and navigate toward reward locations [3]. Our extension introduces a visual channel—processed through a deep convolutional backbone—and combines it with LiDAR data via a fusion layer, enabling the place cell network to form more nuanced cognitive maps. The final output is a robot that can learn and adapt to complex environments with better robustness, mimicking mammalian navigation behavior [6][8].

3.2 Design Objectives

The goal of this project is to extend a biologically inspired navigation model by integrating both LiDAR and visual sensing into a unified spatial representation. The system receives LiDAR scans and camera images, processes them through a vision embedding module and fusion layer, and feeds the resulting features into a neural architecture of place cells and head direction cells. This internal model enables the robot to localize itself and navigate toward goals in complex environments [1][6]. All experiments are conducted in Webots simulation with future deployment intended for iRobot Create 3.

3.2.1 Hardware:

This project does not involve any physical hardware deployment. All development and testing were conducted in simulation using standard laptops, with no reliance on external sensors or robotic platforms.

3.2.2 Software:

Robot and Model:

1. The navigation model is built using Python and PyTorch, simulating biologically inspired components such as place cells and head direction cells [1][2].
2. Visual features are extracted using EfficientNetV2B0, pre-trained on ImageNet and adapted for real-time embedding [6][8].

Simulated Environment:

1. All experiments were conducted using Webots R2025.
2. The robot operates in two types of simulated arena: a simple open square and a complex cross-shaped environment with visual cues.
3. Robot orientation and motion were handled using Webots' native odometry and camera modules. The simulation provided synchronized LiDAR and image data to test the impact of visual fusion on spatial navigation [9].

3.3 Technical Approach

3.3.1 Basic Theory

This model is a bio-inspired reinforcement learning architecture integrating head direction, place cell, and reward cell networks for autonomous navigation. We have augmented the state-space input with visual image information by using a neural network for visual feature extraction, and then fused these features with LiDAR data through a dedicated fusion layer to support navigation. The core algorithm of this model is as follows, enabling the rapid and uniform generation of unsupervised place fields across diverse environments relying solely on boundary inputs:

1. **Place cell competition** ensures that no single area within an environment is overly represented in the layer of place cells, analogous to the distribution of place cells in the hippocampus of mammals. This competition employs a winner-take-all strategy, where the neuron that fires most strongly in response to a specific location inhibits the activity of neighboring neurons. This inhibition compels other place cells to become responsive to different spatial inputs, corresponding to distinct locations. Consequently, place cells distribute themselves evenly across various environmental inputs, stabilizing over time and allowing the place cell network (PCN) to adapt and accurately represent any arbitrary environmental shape.
2. **Synaptic plasticity at the place cell input** enables the place cells that prevail in the competition to strengthen their association with specific place fields. Conversely, place cells that do not win the competition can dissociate from those place fields and form new associations in different areas. This dynamic adaptability ensures that the PCN can continuously reorganize itself based on the robot's interactions with its environment, maintaining an accurate and efficient spatial representation.
3. The **Boundary Vector Cell (BVC) Layer** mimics the mammalian ability to perceive distances to nearby boundaries such as walls or obstacles in multiple directions. In our model, a LiDAR system provides distance measurements directly to the BVC layer, supplying a vector of 720 elements that represent distances evenly distributed across a 360-degree field. This approach maintains simplicity by utilizing LiDAR data directly, avoiding the complexity of replicating biological BVC functions with camera systems.

4. **Head Direction Cells** are another critical component, enabling the model to discern the robot's orientation. These cells exhibit peak activity when the robot aligns with specific, predefined directions. In our implementation, we use eight head direction cells corresponding to eight cardinal directions. The robot's orientation is tracked using simulation data from Webots or odometry messages from ROS2 in the Create3 platform, ensuring accurate alignment and orientation tracking from any starting direction.
5. The **Place Cell Network (PCN)** consists of 1,000 place cells designed to localize the robot within the environment. As the robot navigates and learns about the environment's boundaries, the place cells disperse evenly across the space through the mechanisms of competition and plasticity. This dynamic organization allows the PCN to effectively map the environment, providing robust spatial awareness and facilitating efficient navigation.

The equations for the firing rate of the i^{th} place cell are:

$$\tau_p \frac{ds_i^p}{dt} = -s_i^p + \sum_{j=0}^{n_b} W_{ij}^{pb} v_j^b - \Gamma^{pb} \sum_{j=0}^{n_b} v_j^b - \Gamma^{pp} \sum_{j=0}^{n_p} v_j^p$$

$$v_i^p = \tanh\left(\left[\psi s_i^p\right]_+\right)$$

Overall, the integration of place cell competition, synaptic plasticity, boundary vector inputs, and head direction cells within the PCN framework enables the model to generate a comprehensive and adaptable spatial representation. This foundational theory underpins the model's ability to achieve high navigation efficiency and accurate environmental perception in varied and complex settings.

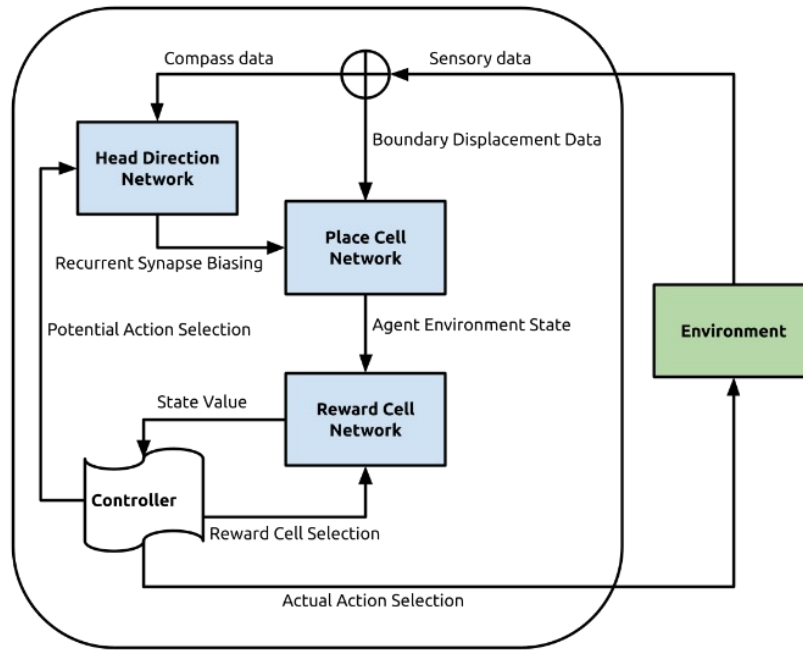


Figure. Overall Model Architecture

we enhance the existing reinforcement learning model by incorporating visual information into the robot's state space to achieve improved navigation efficiency. In the original model, radar data serves as the sole input. This radar data is processed through the Boundary Vector Cell (BVC) layer, generating outputs that feed into the Place Cell Layer (PCL). These outputs function as neural "responses," where neurons that respond most strongly to specific locations inhibit the activity of neighboring neurons. This inhibition compels other place cells to become sensitive to different spatial inputs, corresponding to distinct locations.

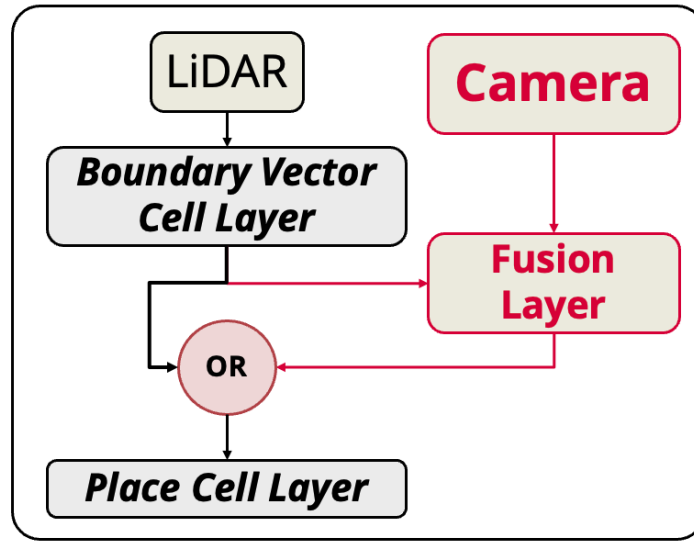


Figure. Vision-Embedding and Fusion Layer

To advance this model, we introduce a camera to simulate the animal's visual signals in addition to the radar data. The visual information is embedded into a dedicated Fusion Layer, where it is integrated with the output signals from the BVC. The fused output is then input into the PCL, effectively embedding visual information into the state space. This multimodal sensory integration aims to enhance the robot's navigation capabilities by leveraging both radar and visual data, thereby enabling more accurate and efficient environmental perception and movement.

3.3.2 Proposed Solution

3.3.2.1 Vision-Embedding

We utilize EfficientNetV2B0, initialized with pre-trained ImageNet weights, as our core visual feature extractor by omitting its top classification layer and employing global average pooling, thus generating a high-level 1280-dimensional feature vector. Specifically, this 1280-

dimensional vector is produced through the final global average pooling operation and subsequently converted into a PyTorch tensor for downstream tasks [8].

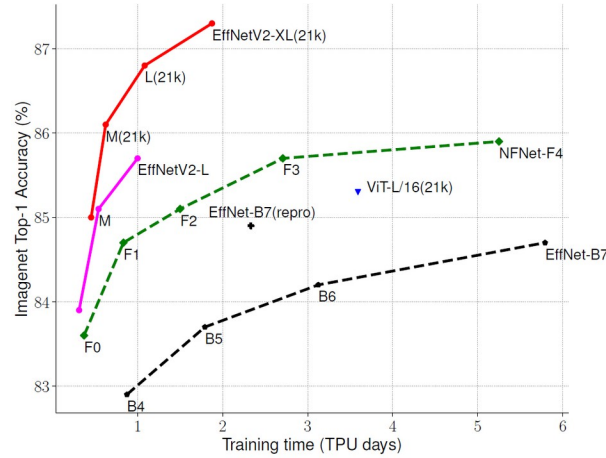


Figure. Training efficiency

EfficientNetV2 represents a cutting-edge convolutional neural network architecture specifically engineered to enhance training and inference efficiency without sacrificing accuracy. Building upon its predecessor EfficientNet, EfficientNetV2 optimizes the network structure and introduces novel training techniques, significantly elevating both speed and performance. A hallmark of EfficientNetV2 is its sophisticated compound scaling method, which meticulously adjusts network depth, width, and resolution to optimize model performance comprehensively. Moreover, it integrates progressive learning strategies and employs optimized convolution operations, notably the innovative Fused-MBConv modules. These modules enhance computational speed, especially when processing large-scale datasets, further solidifying EfficientNetV2's superior performance.

Our implementation of EfficientNetV2 leverages these efficiency-focused innovations effectively. Beginning with an input image of 224x224 pixels, the network initially employs a

conventional 3×3 convolutional layer designed for preliminary feature extraction. Following this initial convolution, two Fused-MBConv4 blocks are introduced. These blocks distinguish EfficientNetV2 from the original EfficientNet architecture by merging the depthwise convolution and the first pointwise convolution into a single standard convolution operation, enhancing performance significantly on contemporary computational accelerators. The notation '4' denotes an expansion ratio of four, indicating the proportional increase in channel dimensionality within these blocks. Subsequently, the architecture incorporates an MBConv6 block, an inverted residual structure typical of EfficientNet variants, characterized by a 3×3 depthwise convolution, squeeze-and-excitation (SE) blocks with a reduction ratio of 0.25, and subsequent pointwise convolutions. Here, the '6' indicates an expansion ratio of six, optimizing channel usage and improving feature representation.

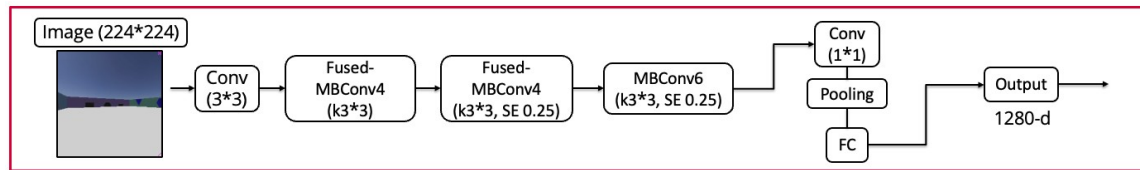


Figure. Vision-Embedding Model

Finally, the network concludes with a global average pooling layer, reducing spatial dimensions and outputting a robust 1280-dimensional feature representation suitable for various visual tasks. This architecture, as illustrated clearly in the accompanying diagram, showcases EfficientNetV2's streamlined yet powerful structure. Its careful design, which emphasizes efficient block configurations and strategic fusion of operations, ensures exceptional performance across diverse visual applications, including image classification, object detection, and semantic segmentation. Furthermore, EfficientNetV2's adaptability makes it exceptionally

suitable for deployment in resource-constrained environments such as mobile devices and edge computing platforms. By balancing computational efficiency with high accuracy, EfficientNetV2 stands out as a versatile and potent solution within the modern computer vision landscape, effectively addressing the growing demand for performance and operational efficiency.

3.3.2.2 Fusion Layer

To effectively integrate visual and boundary vector cell (BVC) information, we introduced a dedicated Fusion Layer leveraging an autoencoder structure built with a three-layer multilayer perceptron (MLP). Initially, we collected data comprising 2000 camera images along with their corresponding BVC outputs within the Webots R2025 testing environment. Each captured image was processed through EfficientNetV2, producing 2000 visual feature vectors, each of dimension 1280. These visual vectors were then concatenated with the corresponding 400-dimensional BVC outputs, resulting in a unified dataset of 2000 input vectors, each 1680-dimensional.

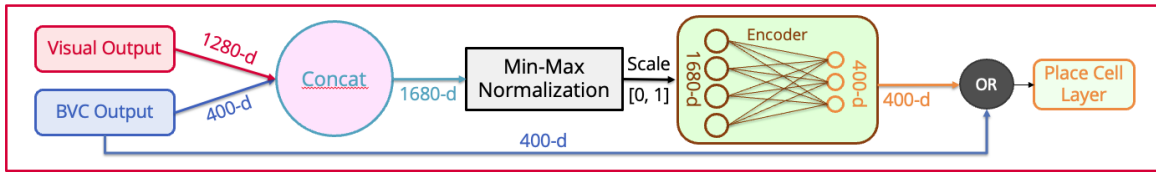


Figure. Fusion Layer

For information fusion, we designed and trained an autoencoder specifically tailored to this concatenated dataset. The architecture of the autoencoder consists of an input layer with 1680 dimensions matching the concatenated vector size, a hidden bottleneck layer with 400 dimensions (corresponding exactly to the original BVC output dimension), and an output layer

restoring the dimensionality back to 1680. After thorough training, the autoencoder parameters were saved as a '.pth' file. Importantly, for practical deployment within the fusion layer, we retained only the parameters from the input to the hidden layer (encoder component), thereby enabling efficient dimensionality reduction and producing a 400-dimensional fused representation matching the original BVC dimension.

$$\tilde{x} = \frac{(x_{BVC} x_{visual}) - \min((x_{BVC} x_{visual}))}{\max((x_{BVC} x_{visual})) - \min((x_{BVC} x_{visual}))}, y = W \tilde{x} + b$$

Operationally, the Fusion Layer functions by first concatenating the visual model's 1280-dimensional output with the BVC model's 400-dimensional output, creating a 1680-dimensional vector. This combined vector undergoes min-max normalization, scaling all features to a uniform range between 0 and 1 to enhance model stability. The normalized tensor is then input into the pretrained encoder, represented by a two-layer MLP with dimensions 1680 to 400.

Mathematically, this encoder operation is expressed as $y = W \tilde{x} + b$, where $\tilde{x}$ denotes the normalized concatenated vector, $W \in R$ is the trained weight matrix of the encoder, and $b \in R$ is the bias vector.

The resulting 400-dimensional tensor from the encoder precisely aligns with the BVC layer's dimensional structure, allowing seamless integration into downstream layers, including the Place Cell layer. This structured fusion approach effectively combines the rich, high-dimensional visual context provided by EfficientNetV2 with the spatial positional accuracy of BVC outputs, significantly enhancing the model's capability to interpret and respond to complex environmental inputs.

3.3.3 Design Alternative

We have prepared three distinct model sizes tailored specifically for the current task, offering flexible solutions to accommodate various application scenarios. The small version of the model is characterized by its low computational cost, making local inference feasible directly on robotic devices with limited computational resources. In contrast, the large version offers significantly enhanced image recognition capabilities suitable for handling complex visual environments, particularly advantageous when high-performance remote computing infrastructure is available. This strategic differentiation ensures optimal performance and operational efficiency across a diverse range of deployment conditions.

Small Model: EfficientNetV2-S is the smallest model in the series, specifically designed for resource-constrained environments. It has fewer parameters and lower computational demands, making it ideal for mobile devices and embedded systems. Despite its compact size, EfficientNetV2-S provides efficient feature extraction capabilities, suitable for real-time image processing and applications requiring rapid responses. Additionally, the small model achieves significantly faster training and inference speeds by utilizing optimized network structures and efficient convolution operations, all while maintaining high accuracy.

EfficientNetV2-S

```
# about base * (width1.4, depth1.8)
v2_s_block = [
    'r2_k3_s1_e1_i24_o24_c1',
    'r4_k3_s2_e4_i24_o48_c1',
    'r4_k3_s2_e4_i48_o64_c1',
    'r6_k3_s2_e4_i64_o128_se0.25',
    'r9_k3_s1_e6_i128_o160_se0.25',
    'r15_k3_s2_e6_i160_o256_se0.25',
]
```

Figure. Small Model

Medium Model: EfficientNetV2-M strikes a balance between the Small and Large models in terms of parameters and computational complexity. It is suitable for applications that demand higher performance and efficiency but still operate within limited resources. The Medium model delivers robust performance in tasks such as image classification and object detection while maintaining good computational efficiency. This makes it an ideal choice for applications that lie between mobile devices and server-side deployments, offering enhanced capabilities without excessive resource consumption.

EfficientNetV2-M

```
# about base * (width1.6, depth2.2)
v2_m_block = [
    'r3_k3_s1_e1_i24_o24_c1',
    'r5_k3_s2_e4_i24_o48_c1',
    'r5_k3_s2_e4_i48_o80_c1',
    'r7_k3_s2_e4_i80_o160_se0.25',
    'r14_k3_s1_e6_i160_o176_se0.25',
    'r18_k3_s2_e6_i176_o304_se0.25',
    'r5_k3_s1_e6_i304_o512_se0.25',
]
```

Figure. Medium Model

Large Model: EfficientNetV2-L is the largest variant, featuring the highest number of parameters and computational complexity to achieve optimal performance. It is designed for applications that require handling high-resolution images and complex visual tasks, such as high-precision image recognition, video analysis, and advanced computer vision research. Although it demands more computational resources, EfficientNetV2-L excels in large-scale datasets, capturing more detailed features and improving overall model accuracy and robustness.

EfficientNetV2-L

```
# about base * (width2.0, depth3.1)
v2_l_block = [
    'r4_k3_s1_e1_i32_o32_c1',
    'r7_k3_s2_e4_i32_o64_c1',
    'r7_k3_s2_e4_i64_o96_c1',
    'r10_k3_s2_e4_i96_o192_se0.25',
    'r19_k3_s1_e6_i192_o224_se0.25',
    'r25_k3_s2_e6_i224_o384_se0.25',
    'r7_k3_s1_e6_i384_o640_se0.25',
]
```

Figure. Large Model

By offering these three different sizes, the EfficientNetV2 series provides flexibility to address a wide range of application scenarios, from resource-limited mobile devices to high-performance servers and research platforms, effectively meeting varying levels of demand.

3.3.4 Design Considerations

- **Computational Efficiency:** Consider employing efficient neural network models (like EfficientNetV2) for visual processing to balance high performance with manageable computational loads, enabling deployment on feasible hardware (including potential onboard processing).
- **Optimized Information Integration:** Consider developing a dedicated fusion mechanism (e.g., autoencoder-based layer) to effectively combine diverse sensor data (spatial and visual) into a cohesive internal state, aiming for more accurate localization and decision-making.

- **Deployment Flexibility:** Consider offering scalable model variants (e.g., Small, Medium, Large) to provide stakeholders with options that balance performance requirements against available computational resources and cost constraints.

3.3.5 Design Constraints Identification

- **Hardware Costs:** The cost of LiDAR, cameras, and computational hardware (potentially GPUs for EfficientNetV2) limits the choice of components. The provision of scalable models (Small, Medium, Large) directly addresses balancing cost and performance.
- **Development Costs:** Limited time/resources may constrain the complexity of algorithms explored or the extent of testing. Using pre-trained models (EfficientNetV2) is a strategy to mitigate this.
- **Legal and Regulatory Considerations:** Deployment would be constrained by data privacy laws (e.g., GDPR), robotics safety standards, and potentially other domain-specific regulations.

3.3.6 Engineering standards and Codes Applied to the Project

- **Code Style Standardization:** All Python components, including the simulation core, control systems, and visualization tools, are developed in accordance with the PEP 8 style guide to promote code consistency, readability, and maintainability throughout the project codebase.
- **Version Control Methodology:** The project employs Git for version control, adhering to the Gitflow branching model. This workflow facilitates systematic development through the use of feature branches for isolated work, mandatory pull requests coupled with code

reviews for quality assurance, and semantic versioning for structured release management and tracking.

- **Semantic Interoperability Consideration:** The system's core neural components (e.g., place cells, reward cells, head direction cells) are designed with consideration for mapping to standard robotics and automation ontologies, such as IEEE 1872. This foresight aims to enable potential future semantic interoperability with other compatible robotic systems or platforms.

3.3.7 Process

The work of this project is organized into three main stages: (1) simulation environment construction, (2) core algorithm implementation, and (3) data testing and analysis.

1) Simulation Environment Construction

In this stage, we designed two variants of a test arena in Webots R2025: “Easy” and “Complex.”

- **Easy Environment:** a $10\text{ m} \times 10\text{ m}$ square room whose four walls are each painted a different color and decorated with distinct patterns to maximize visual discriminability.
- **Complex Environment:** built on the Easy layout by placing a cross-shaped obstacle at the room’s center, intended to induce severe spatial aliasing and thus evaluate the robot’s robustness under challenging conditions.

A forward-facing camera module was mounted on the robot, providing a visual input stream at 224×224 pixel resolution aligned with the robot’s heading.

2) Core Algorithm Implementation

This stage is divided into two subtasks: the vision model and the Fusion Layer

- **Vision Model:** each camera frame is treated as a 3D tensor of size $224 \times 224 \times 3$, with 8-bit RGB channels (values 0–255). We employ a pre-trained EfficientNetV2 backbone and extract the activations of its penultimate layer to represent high-level visual features.
- **Fusion Layer:** the 400-dimensional feature vector from the vision model is concatenated with the output of the Boundary Vector Cell (BVC) layer. This combined vector then undergoes normalization and is processed by an encoder module, resulting in a fused, 400-dimensional representation that encodes both visual and spatial information.

3) Data Testing and Analysis

In the final stage, we use a logical OR gate to enable or disable the vision model dynamically.

We then conduct experiments in both the Easy and Complex environments to assess how the inclusion of visual inputs influences the activation patterns of the place-cell layer.

3.3.8 Performance

3.3.8.1 Experimental Setting

The test environment employs a 10×10 world, with each of the four walls featuring a unique color and pattern. In complex settings, we additionally place a cross in the center to introduce interference for the robot.

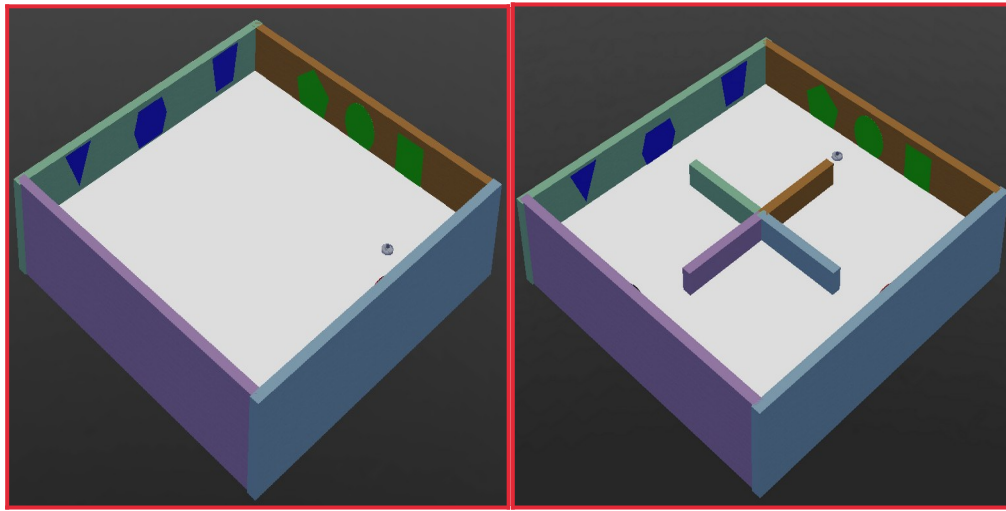


Figure. Easy (left) and Complex (right) Environment

Parameter	Value	Description
Place Cells	500	Number of neurons in the place cell network.
BVCs	50	Number of BVCs per direction.
Head Direction	8	Number of head direction cells.
Time Step (ms)	96	Duration of each simulation step (32×3).
Radial Sigma	0.5	Standard deviation for the radial component.

Table. Experimental Parameters Setting

The table above presents the core parameter configurations of the experimental environments along with their respective descriptions.

3.3.8.2 Results

We conducted experiments with the visual channel both enabled and disabled.

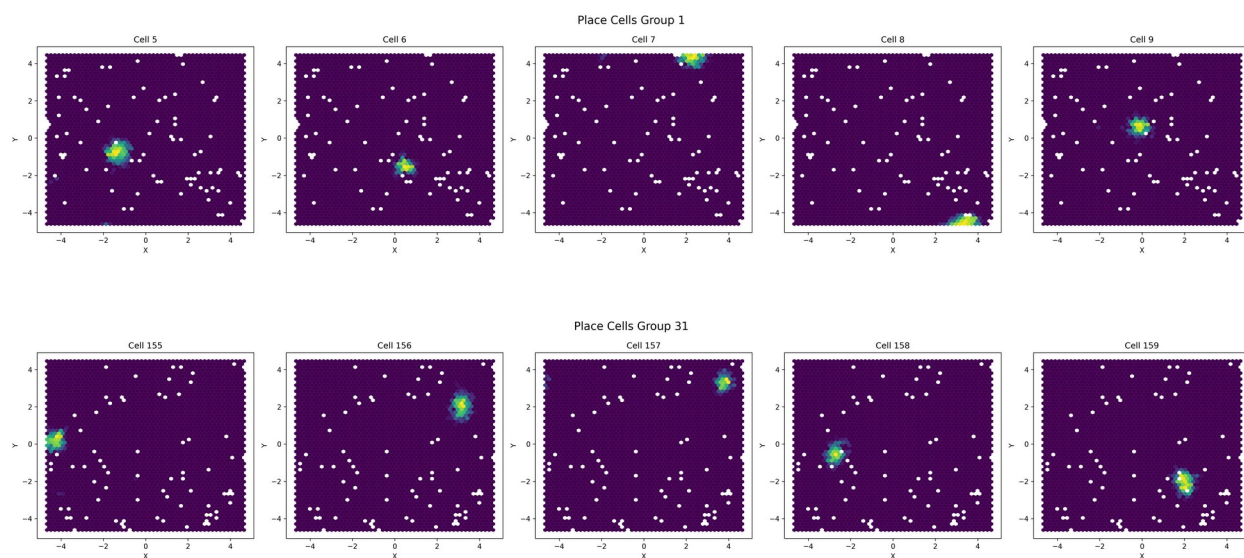


Figure. Place Cell Field Group under Enable Vision Embedding

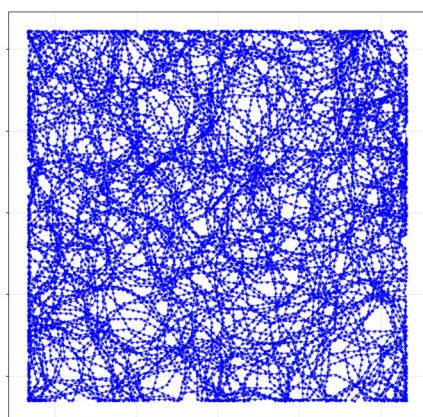


Figure. Agent Path under Enable Vision Embedding

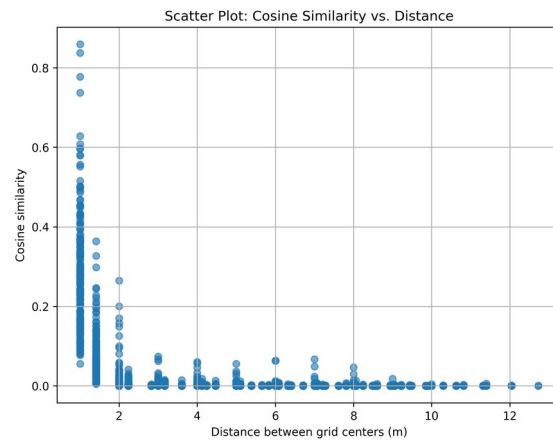


Figure. Cosine Similarity under Enable Vision Embedding

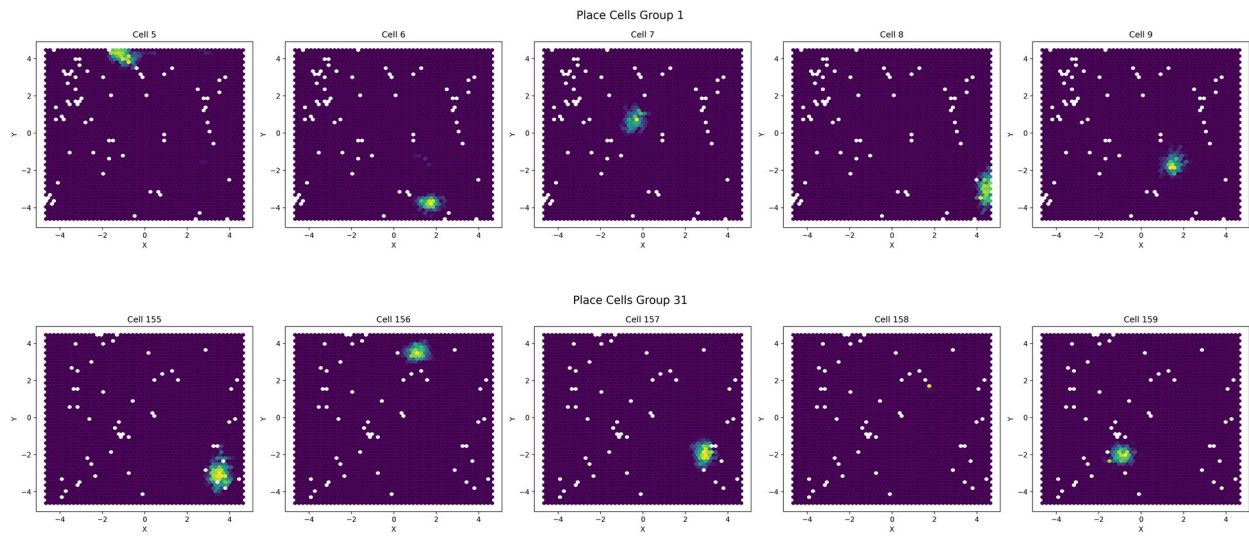


Figure. Place Cell Field Group under Disable Vision Embedding

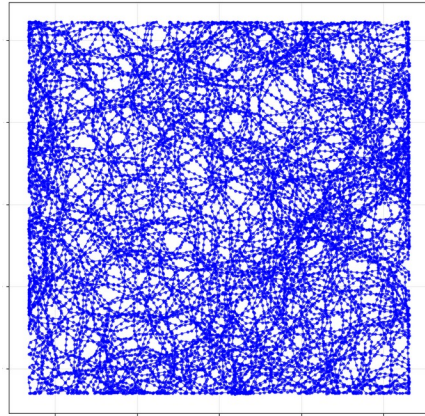


Figure. Agent Path under Disable Vision Embedding

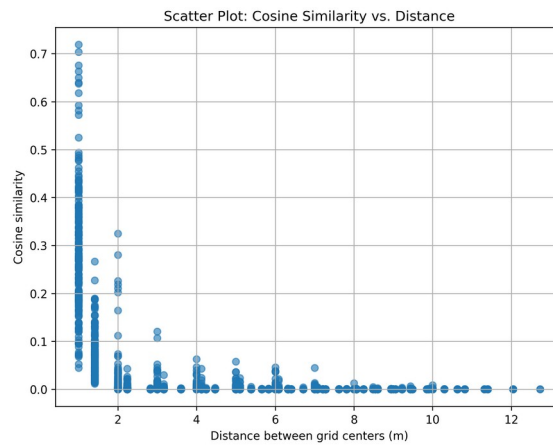


Figure. Cosine Similarity under Disable Vision Embedding

In the Easy environment, enabling visual input produced a modest improvement in the smoothness of the cosine-similarity distribution: as grid separation increased, cosine similarity declined more uniformly, indicating a more homogeneous activation pattern of place cells. This subtle enhancement demonstrates that incorporating visual cues can marginally refine spatial representation.

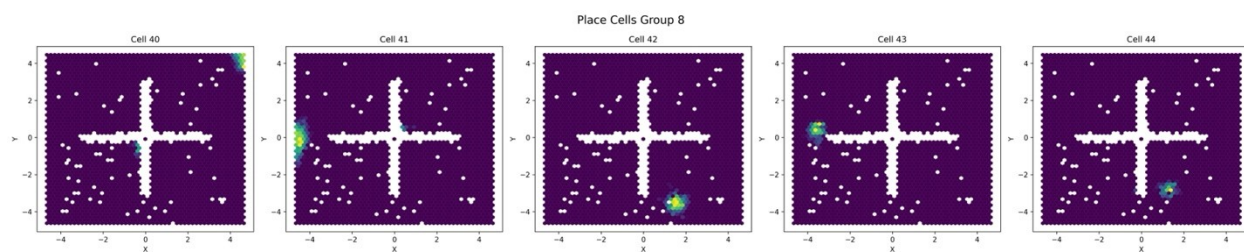


Figure. Place Cell Field Group under Enable Vision Embedding

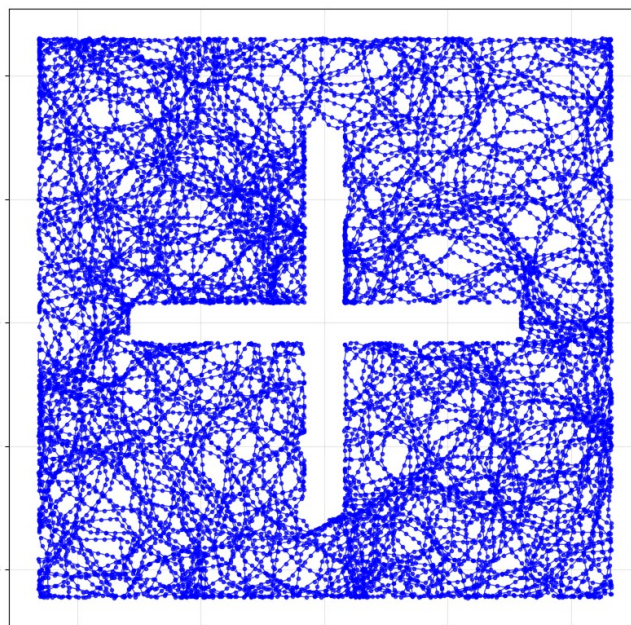


Figure. Agent Path under Enable Vision Embedding

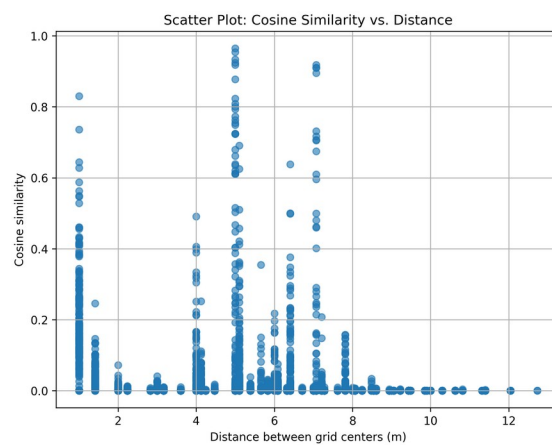


Figure. Cosine Similarity under Enable Vision Embedding

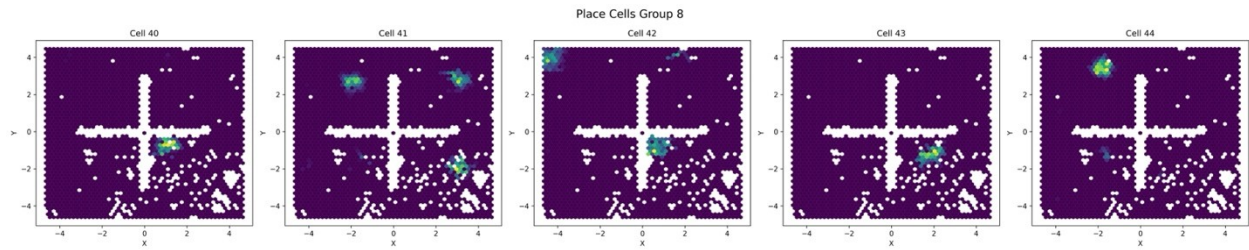


Figure. Place Cell Field Group under Disable Vision Embedding

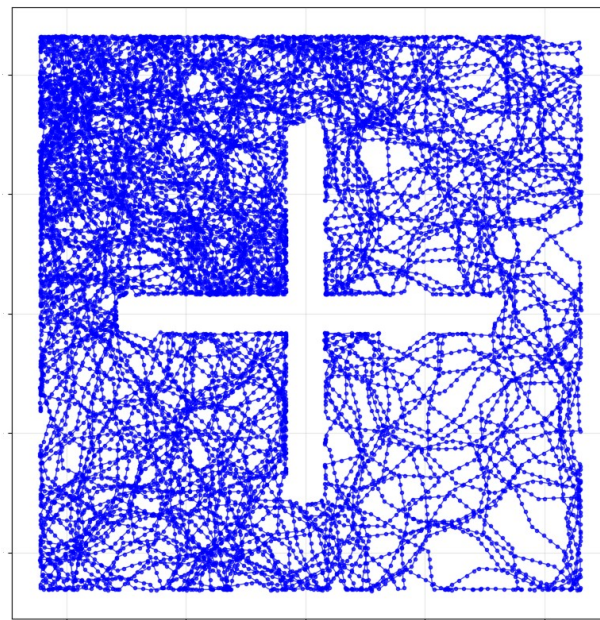


Figure. Agent Path under Disable Vision Embedding

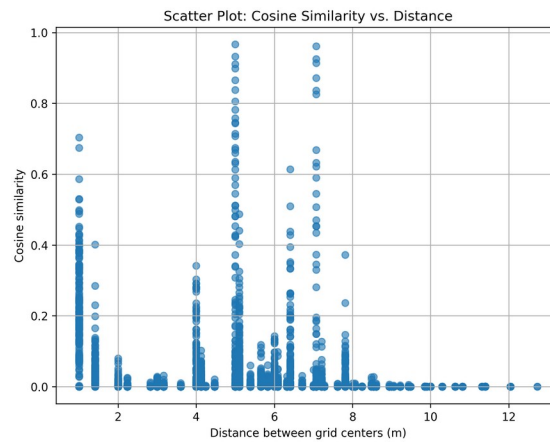


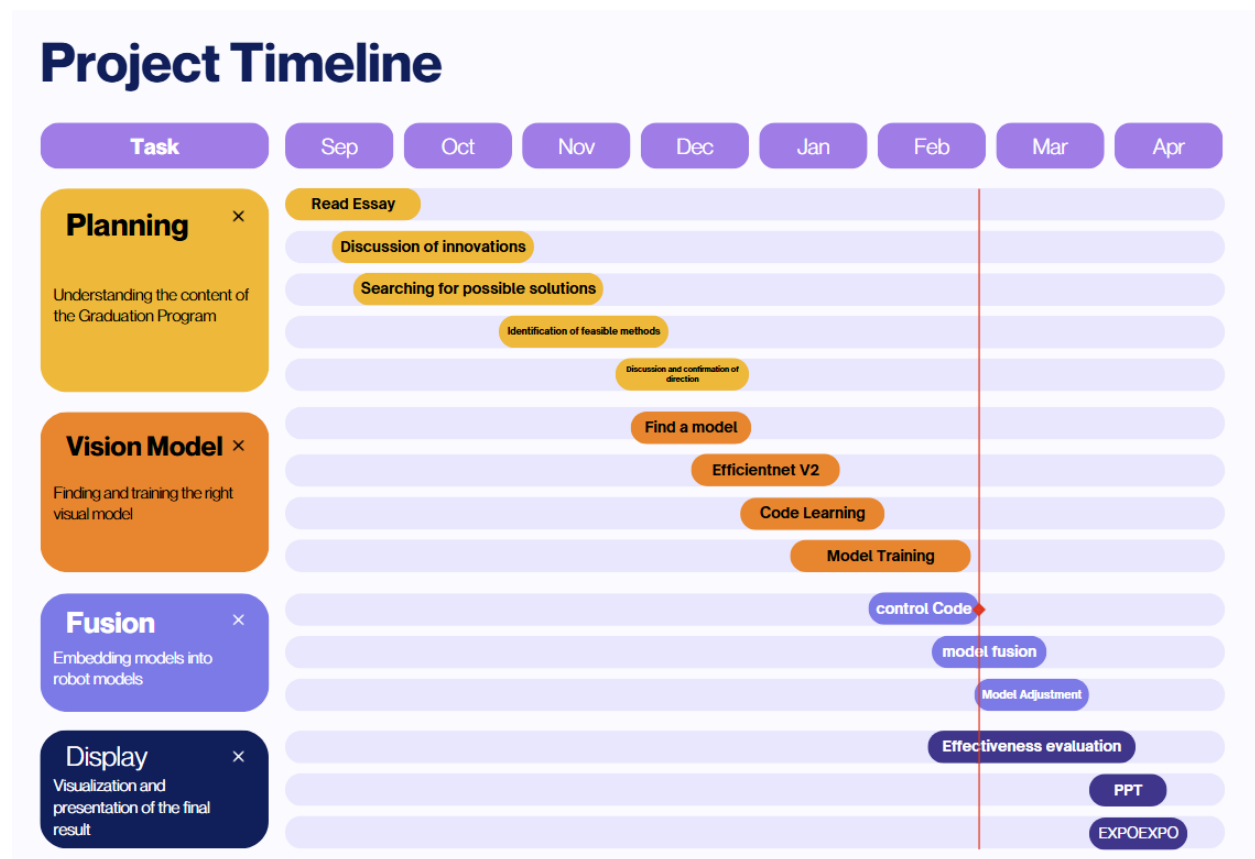
Figure. Cosine Similarity under Disable Vision Embedding

In the Complex environment, the results shown in the preceding figure present some instructive pictures. When the visual model is enabled, the robot's exploration trajectory becomes more uniformly distributed, whereas with vision disabled it spends a disproportionate amount of time stalled in the upper-left quadrant. Correspondingly, place-cell activations under visual guidance appear more tightly clustered within certain cell groups, suggesting a localized benefit in spatial encoding. However, these apparent improvements are not broadly representative: cosine-similarity analysis continues to reveal pronounced spatial aliasing, as evidenced by an unexpected spike in similarity for grid separations of 4–8 m, indicating that distinct locations still evoke nearly identical feature representations. Although the decline of similarity over shorter distances (1–4 m) is marginally smoother when visual input is incorporated, the fundamental aliasing interference remains unmitigated, demonstrating that the visual augmentation yields only a minor smoothing effect at close range.

These findings underscore the limited contribution of the EfficientNetV2B0 backbone to overall spatial discriminability. Although this visual backbone supplies supplementary semantic and texture information and can approximate LiDAR responses within localized regions, our experiments demonstrate that it lacks the sensitivity to spatial variation required to differentiate geometrically distinct but visually analogous areas. In highly symmetric layouts—such as the cross-shaped arena—certain place cells were activated repeatedly at multiple, spatially separated sites, and physically distant positions often generated nearly identical feature vectors. Despite the high-contrast textures and distinct pattern variations in the test arena, these limitations endured across all trials. In the absence of explicit viewpoint encoding or mechanisms that enforce spatial

continuity, this shortcoming gives rise to persistent perceptual aliasing in structurally repetitive settings, limiting the agent’s ability to resolve real-world spatial ambiguities. Taken together, these results indicate that while visual cues can slightly refine local spatial coding, they fail to overcome fundamental limitations of place-cell based mapping in repetitive environments where structural symmetry prevails.

3.4 Timeline



1. Planning Phase (September – January)

Objective: Define scope, survey the literature, and finalize research direction

- **September:** Read key essays and papers to understand the graduation project requirements.
- **October:** Hold team brainstorming sessions to explore innovative vision–LiDAR fusion ideas.
- **November:** Research and shortlist several feasible technical solutions.
- **December:** Compare and evaluate methods; confirm our overall direction and technical approach.
- **January:** Write and submit the detailed project proposal, including deliverables and milestone assignments.

2. Vision Model Phase (December – March)

Objective: Select, implement, and train the visual feature extractor

- **Late December:** Survey state-of-the-art CNN backbones and decide on EfficientNet V2.
- **January:** Build the EfficientNet V2 feature-extraction pipeline and integrate it with our robot codebase.
- **February:** Develop and refine all vision-model scripts, ensuring data preprocessing, augmentation, and model-inference interfaces are stable.
- **February – March:** Train and fine-tune the model on our dataset, producing robust 1 280-dimensional image embeddings.

3. Fusion Phase (February – April)

Objective: Merge LiDAR-based BVC outputs with visual embeddings into a unified feature vector

- **February:** Complete core control code and reserve APIs for the fusion layer.

- **March:** Implement the fusion module: Concatenate 400-D BVC output with 1 280-D visual features $\rightarrow$ 1 680-D tensor Min–Max normalize this tensor Encode into a 400-D fused feature vector via a pretrained encoder
- **Late March:** Tune fusion-layer hyperparameters based on simulation feedback to improve spatial discriminability.

4. Presentation Phase (March – April)

Objective: Evaluate performance, document findings, and prepare final deliverables

- **Mid–Late March:** Run extensive tests in both simple and cross-shaped (complex) environments; measure navigation smoothness and place-cell activation accuracy.
- **End of March – Early April:** Draft the lab report and create the project PPT, complete with heatmaps, cosine-similarity curves, and quantitative metrics.
- **April:** Present our bio-inspired, vision-fused navigation system at the EXPO Expo, showcasing the robot’s enhanced spatial understanding.

3.5 Problems Encountered / Analysis of Problems Solved

1. Feature–Dimension & Scale Mismatch

Problem: Directly concatenating the 400-D BVC tensor with the 1 280-D visual embedding produced a 1 680-D feature vector whose components lay on very different numerical scales. This led to unstable fusion training and slow convergence.

Solution: We applied min–max normalization to the 1 680-D tensor and then used a lightweight encoder (pretrained and fine-tuned) to project it back into a 400-dimensional space. This aligned the feature scales and drastically improved the stability and speed of downstream learning.

2. Fusion-Layer Integration Overhead

Problem: Embedding EfficientNetV2 inference and the fusion computations directly into the Webots control loop caused non-negligible latency, risking missed simulation time steps.

Solution: We refactored the control code into a modular API, decoupling vision inference, normalization, and fusion. By running vision encoding asynchronously and buffering fused vectors, we maintained real-time performance without dropping frames.

3. Perceptual Aliasing in Symmetric Environments

Problem: In the cross-shaped arena, several place cells activated at multiple, geometrically distinct locations—i.e. visual embeddings lacked spatial sensitivity, producing high cosine similarity for distant grid points.

Solution: Through A/B testing (vision-enabled vs. vision-disabled), we quantified the aliasing effect and then tuned fusion-layer hyperparameters (learning rate, encoder depth). Although aliasing was not eliminated, these adjustments yielded a modest smoothing of the cosine-similarity curve, demonstrating the fusion mechanism’s partial success.

4. Limited Benefit in Simple Layouts

Problem: In obstacle-free environments, LiDAR alone already yields near-perfect navigation; adding visual features did not consistently improve path smoothness or reward propagation.

Solution: We integrated automated metrics (heatmap overlap, reward activation statistics) into our evaluation pipeline. This allowed us to detect when vision cues were redundant, focus our tuning on complex scenarios, and avoid overfitting fusion parameters to trivial cases.

3.6 Future Recommendations

Although our use of EfficientNetV2B0 as a visual backbone adds valuable semantic and texture cues, experiments demonstrate its limited role in improving overall spatial discriminability. In structurally symmetric settings—such as the cross-shaped arena—certain place cells fire at multiple, geometrically distinct locations, and cosine-similarity metrics reveal that visually similar but spatially distant grid points remain indistinguishable. This perceptual aliasing arises from the lack of explicit viewpoint encoding and spatial continuity within the visual pipeline.

To overcome these limitations and fully leverage visual input in a multimodal navigation system, we recommend the following directions:

1. Adopt Vision Models with Enhanced Spatial Sensitivity

Investigate architectures designed for localization tasks—e.g., models incorporating positional embeddings or spatial attention—to ensure visual features carry stronger geometry-aware signals.

2. Incorporate Multi-View Image Encoding

Capture each location from multiple perspectives (e.g., rotating camera viewpoints or stereo rigs) to enrich the embedding space with viewpoint-dependent variations, reducing perceptual aliasing.

3. Leverage Cross-Modal Contrastive Learning

Train the fusion encoder with a contrastive objective that explicitly aligns LiDAR and visual feature spaces, encouraging discriminative representations even in visually repetitive areas.

4. Develop Deeper, Attention-Based Fusion Networks

Move beyond simple concatenation and pooling by integrating multi-head attention or transformer-style modules that dynamically weight and integrate cues from each modality.

By pursuing these strategies, future iterations of our bio-inspired navigation framework can strengthen the complementary strengths of vision and LiDAR, yielding more robust spatial representations and more reliable goal-directed behaviors.

4 Conclusion

In this work, we systematically investigated the limits and benefits of fusing visual semantics with LiDAR-based boundary sensing in a bio-inspired navigation framework. By embedding an EfficientNetV2B0 backbone alongside boundary vector cells and head-direction networks, we demonstrated that visual features can enrich semantic context without degrading geometric accuracy. However, our experiments in symmetric and complex environments revealed that naïve concatenation and normalization still suffer from perceptual aliasing, as visually similar yet spatially distinct locations remain difficult to distinguish.

The proposed fusion layer—combining min–max normalized BVC outputs with 1 280-D image embeddings and encoding back into a unified 400-D vector—proved effective at stabilizing training and integrating multi-sensory cues in real time. Although improvements in navigation smoothness and place-cell discrimination were modest, our framework lays the groundwork for

deeper cross-modal alignment and attention-based fusion strategies. Ultimately, this study highlights that while vision serves as a powerful semantic enhancer, unlocking its full potential for spatial encoding will require specialized architectures, multi-view representations, and contrastive learning objectives tailored to the challenges of robotic navigation.

References

- [1] Alabi, A., Vanderelst, D., and Minai, A. A. "Rapid Learning of Spatial Representations for Goal-Directed Navigation Based on a Novel Model of Hippocampal Place Fields." *Neural Networks*, vol. 161, 2023, pp. 116–128.
- [2] Alabi, A., Vanderelst, D., and Minai, A. A. "Context-Dependent Spatial Representations in the Hippocampus Using Place Cell Dendritic Computation." *Proceedings of the IEEE World Congress on Computational Intelligence (WCCI)*, Padua, Italy, July 2022, pp. 1–10.
- [3] Gerstenslager, A., and Dukenbaev, B. "Using a Simulated Place-Cell Model for Autonomous Reward Mapping and Navigation on a Robotic Platform." Bachelor's thesis, Department of Electrical Engineering and Computer Science, University of Cincinnati, Apr. 2024.
- [4] Cutsuridis, V., and Wennekers, T. "Hippocampus, Microcircuits and Associative Memory." *Neural Networks*, vol. 22, no. 8, 2009, pp. 1120–1128.
- [5] O'Keefe, J., and Dostrovsky, J. "The Hippocampus as a Spatial Map: Preliminary Evidence from Unit Activity in the Freely Moving Rat." *Brain Research*, vol. 34, no. 1, 1971, pp. 171–175.
- [6] Tan, M., and Le, Q. V. "EfficientNetV2: Smaller Models and Faster Training." *arXiv*, arXiv:2104.00298, Jun. 2021, <https://arxiv.org/abs/2104.00298>.
- [7] Elsken, T., Metzen, J. H., and Hutter, F. "Neural Architecture Search: A Survey." *Journal of Machine Learning Research*, 2019.
- [8] Huang, G., Liu, Z., Van Der Maaten, L., and Weinberger, K. Q. "Densely Connected Convolutional Networks." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

- [9] Pandey, Krishna Kant, Prases K. Mohanty, and Dayal R. Parhi. "Real Time Navigation Strategies for Webots Using Fuzzy Controller." 2014 IEEE 8th International Conference on Intelligent Systems and Control (ISCO), IEEE, 2014.
- [10] Bello, I. *LambdaNetworks: Modeling Long-Range Interactions without Attention*. *International Conference on Learning Representations (ICLR)*, 2021M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," Jun. 23, 2021, *arXiv*:
- [11] Michel, Olivier. "Cyberbotics Ltd. Webots™: Professional Mobile Robot Simulation." *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, 2004, pp. 5.